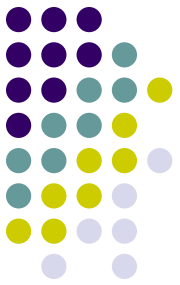


IV Sem(2025-26)
Artificial Intelligence
(24CS01TH0403)

Dr. Pravin M. Sonsare
Assistant Professor
Computer Science and Engineering
RBU, Nagpur

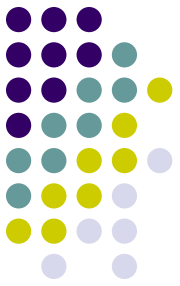




Course Objectives:

- Introduction of problem-solving techniques, task domains and intelligent agent structures in AI.
- Representation of given problem using state space representation and solve it by using different search techniques.
- Understanding of adversarial search, game-playing strategies and constraint satisfaction problems.
- Understand of knowledge representation and uncertainty theory in designing AI systems.

Course Contents



Unit 1. Introduction to AI

Unit 2. Uninformed search techniques

Unit 3. Informed search techniques

Unit 4. Adversarial Search

**Unit 5. Knowledge Representation and
Uncertainty theory**

Books



Books:

- Stuart Russel and Peter Norvig; Artificial Intelligence: A Modern Approach; Third Edition; Pearson Education, 2009.
- E. Rich, K. Knight, S. B. Nair; Artificial Intelligence; 3rd Edition; Tata McGraw Hill, 2014.

Course Outcomes:



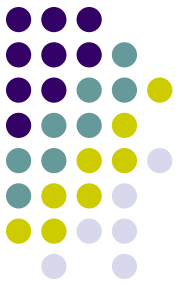
On completion of the course, student will be able to

CO1: Explain the historical evolution, applications and problem-solving characteristics of AI.

CO2: Apply uninformed and informed search techniques and represent given problem using state space representation.

CO3: Utilize different AI techniques to solve fully informed two player games and constraint satisfaction problems.

CO4: Demonstrate knowledge representation techniques and Uncertainty theory in AI decision making scenarios.



UNIT1:Introduction

➤ Contents

- Basics of problem solving
- Problem representation (toy problems and real world problems)
- Structure of agent, rational agent, Specifying task environment
- Properties of task environment
- Measuring problem-solving performance

What is artificial intelligence?



There are no clear consensus on the definition of AI

Q. What is artificial intelligence?

A. It is the science and engineering of making intelligent machines, especially intelligent computer programs. It is related to the similar task of using computers to understand human intelligence, but **AI does not have to confine itself to methods that are biologically observable.**

Q. Yes, but what is intelligence?

A. Intelligence is the computational part of the ability to achieve goals in the world. Varying kinds and degrees of intelligence occur in people, many animals and some machines.

Other possible AI definitions

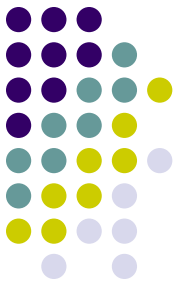


- AI is a collection of hard problems which can be solved by humans and other living things, but for which we don't have good algorithms for solving.
 - e. g., understanding spoken natural language, medical diagnosis, learning, self-adaptation, reasoning, chess playing, proving math theories, etc.

System that Act like human (Turing test)

- Natural Language processing for communication
- Knowledge representation to store information
- Automated Reasoning to use stored information, To answer questions and draw new conclusions
- Machine Learning to adapt to new circumstances and to detect and extrapolate (to infer) patterns
- Computer Vision: To perceive objects
- Robotics: To manipulate objects and move about

Other possible AI definitions



System that Think like human (human-like patterns of thinking steps)

- Through Introspection trying to catch our own thoughts as they go by.
- Through psychological experiments.

-Once we have a sufficiently precise theory of the mind it becomes possible to express the theory as a computer program.

System that think rationally (logically, correctly)

- Aristotle was one of the first to attempt to codify “right Thinking” which is called **Syllogisms**
- patterns for argument structures with premises and conclusions.
- **Example** Socrates is a man
- all men are mortal
- therefore Socrates is mortal

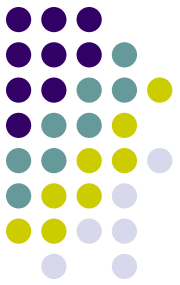
Other possible AI definitions



- The field of logic developed along with programs to solve all kinds of problem statements using logical notations.
- It stresses on correct inference.
- Obstacles:
 - Difficult to take informal knowledge and state it in formal logical notations.
 - Facts about knowledge are quite extensive which can exhaust computational resources.

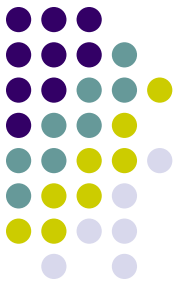
System that Acts Rationally acting so as to achieve one's goal given one's beliefs.

- An agent: It is an entity that acts.
- Rational agent acts so as to achieve best outcome.
- This approach is more general than the three approaches and has most properties of other three approaches included in it.
- This approach is more scientific than approaches based on human thought as rationality can be clearly defined.
- However, achieving perfect rationality is not the aim of AI as it requires excessive computational resources. Limited rationality is adopted for complicated environments.



The rational agent approach: Few highlights

- The rational-agent approach to AI has prevailed throughout most of the AI field's history. In the early decades, rational agents were built on logical foundations and formed definite plans to achieve specific goals.
- Later, methods based on probability theory and machine learning allowed the creation of agents that could make decisions under uncertainty to attain the best expected outcome.
- In a nutshell, AI has focused on the study and construction of agents that do the right thing.
- What counts as the right thing is defined by the objective that we provide to the agent.



Finally AI....

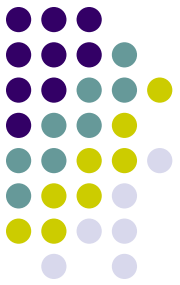
What is AI

It can be defined as: Machines performing as good as humans, one that systemizes and automates intellectual task etc. The definitions vary according to applications

- It can be defined along two main dimensions
 - Thought processes, rationality and reasoning (top)
 - Acting or [behavior] (bottom)
-
- The left side definitions measure success in terms of human performance
 - The right side measure the success in terms of rationality (ideal concept of human intelligence)

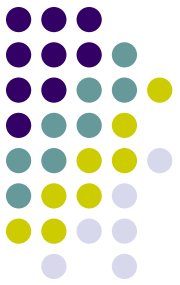
1. Systems that think like humans	3. Systems that think rationally
The new effort to make computers think (machines with mind)	The study of computations that make it possible to perceive reason and act
2. Systems that act like humans	4. Systems that act rationally
The study of how to make computers do things which currently people do better	It is the study of design of intelligent agents

AI Concepts

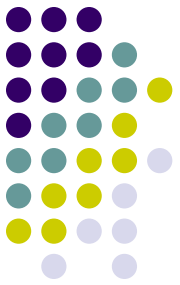


- All four approaches to AI have been followed.
- There is always a tension between human centered approach and a rationalist approach.
- Human centered approach is based on empirical science, involves hypothesis and experimental confirmation.
- A system is rational if it does a “**right thing**”, given what it knows.
- A rationalist approach involves a combination of mathematics and engineering.

History of AI



- AI has roots in a number of scientific disciplines
 - computer science and engineering (hardware and software)
 - philosophy (rules of reasoning)
 - mathematics (logic, algorithms, optimization)
 - cognitive science and psychology (modeling high level human/animal thinking)
 - neural science (model low level human/animal brain activity)
 - Linguistics, economics, etc.
- The birth of AI (1943 – 1956)
 - Pitts and McCulloch (1943): simplified mathematical model of neurons (resting/firing states) can realize all propositional logic primitives (can compute all Turing computable functions)
 - Allen Turing: Turing machine and Turing test (1950)
 - Claude Shannon: information theory; possibility of chess playing computers

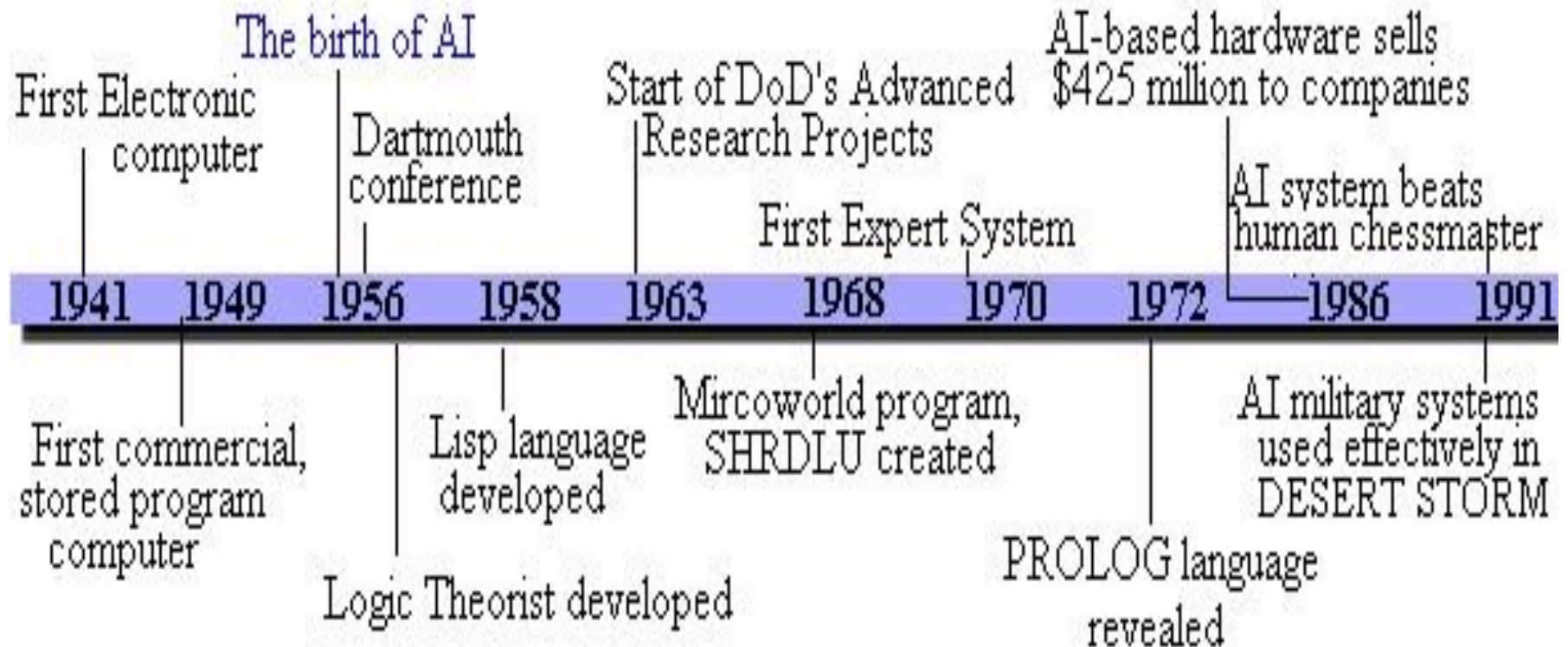
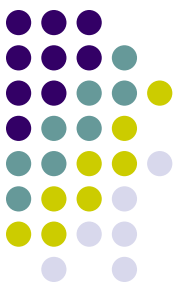


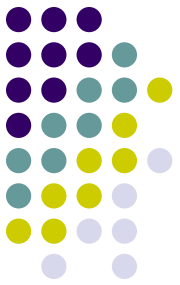
- Early enthusiasm (1952 – 1969)
 - 1956 Dartmouth conference
John McCarthy (Lisp);
Marvin Minsky (first neural network machine);
Alan Newell and Herbert Simon (GPS) General Problem Solver;
 - Emphasize on intelligent general problem solving
GSP (means-ends analysis);
Lisp (AI programming language);
Resolution by John Robinson (basis for automatic theorem proving);
heuristic search (A*, AO*, game tree search)
- Emphasis on knowledge (1966 – 1974)
 - Domain specific knowledge is the key to overcome existing difficulties
 - Knowledge representation (KR) paradigms
 - Declarative vs. procedural representation



- Knowledge-based systems (1969 – 1979)
 - DENDRAL: the first knowledge intensive system (determining 3D structures of complex chemical compounds)
 - MYCIN: first rule-based expert system (containing 450 rules for diagnosing blood infectious diseases)
EMYCIN: an ES shell
 - PROSPECTOR: first knowledge-based system that made significant profit (geological ES for mineral deposits)
- AI became an industry (1980 – 1989)
 - wide applications in various domains
 - commercially available tools
- Current trends (1990 – present)
 - more realistic goals
 - more practical (application oriented)
 - distributed AI and intelligent software agents
 - resurgence of neural networks and emergence of genetic algorithms

History



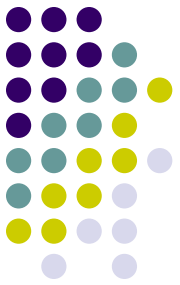


Task Domains

Task categories:

- Mundane Tasks
 - Formal Tasks
 - Expert Tasks
- A person who knows how to perform tasks from the above **categories learns the necessary skills in a standard order**. First, perceptual, linguistic and commonsense skills are learned. Later on, the formal and expert tasks are learnt.
 - Expert skills require specialized domain knowledge, but they often require much less knowledge than mundane skills. The knowledge required for expert skills is also easier to represent and handle inside a program. Such programs are **called expert systems**. Three to four decades back **MYCIN** in the medical domain and **TAXMAN** in the Financial domain were developed as rule based expert systems.

Task Domains of AI



Mundane Tasks

- Perception
 - Vision
 - Speech
- Natural language
 - Understanding
 - Generation
 - Translation
- Commonsense reasoning
- Robot control

Formal Tasks

- Games
 - Chess
 - Backgammon
 - Checkers -Go
- Mathematics
 - Geometry
 - Logic
 - Integral calculus
 - Proving properties of programs

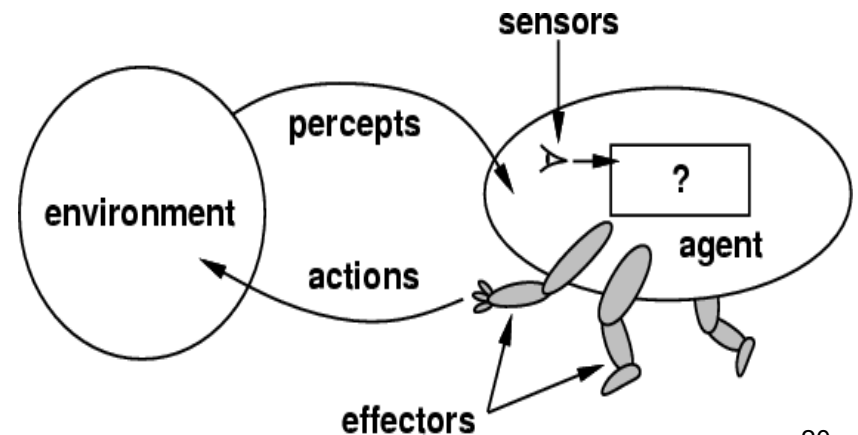
Expert Tasks

- Engineering
 - Design
 - Fault finding
 - Manufacturing planning
- Scientific analysis
- Medical diagnosis
- Financial analysis

Intelligent Agents



- Definition: An **Intelligent Agent** perceives its environment via **sensors** and acts rationally upon that environment with its **actuators**.
- In general, *an agent's choice of action at any given instant can depend on the entire percept sequence observed to date, but not on anything it hasn't perceived*
 - Autonomous
 - Interacts with other agents plus the environment
 - Reactive to the environment
 - Pro-active (goal- directed)
 - Future problems



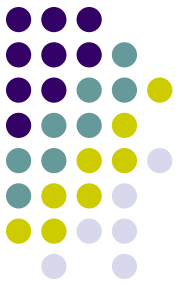


Sensors/Percepts and Actuators/actions?

- Humans

- Sensors: Eyes (vision), ears (hearing), skin (touch), tongue (gustation), nose (olfaction).
- Percepts:
 - At the lowest level – electrical signals from these sensors
 - After preprocessing – objects in the visual field (location, textures, colors, ...), auditory streams (pitch, loudness, direction), ...
- Actuators: limbs, digits, eyes, tongue, ...
- Actions: lift a finger, turn left, walk, run, carry an object, ...
- The Point: percepts and actions need to be carefully defined, possibly at different levels of abstraction
- An agent's behavior is described by the agent function that maps any given percept sequence to an action.

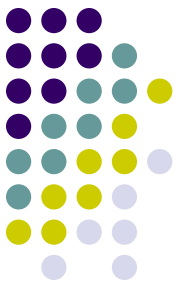
For e.g. A software agent receives keystrokes, file contents, network packets (Percepts) and acts on the environment by displaying output on screen, writing to files or sending packets (Actions).



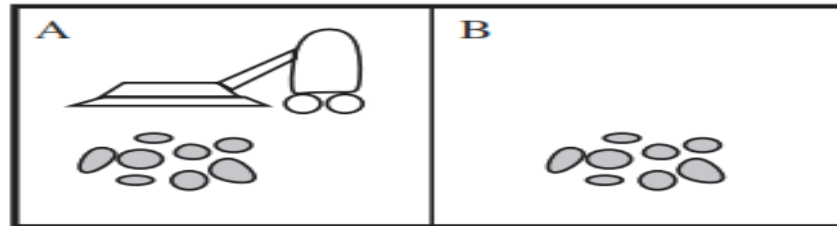
- The agent function is an abstract mathematical description; the agent program is a concrete implementation, running within some physical system.

```
function SKELETON-AGENT(percept) returns action
  static: memory, the agent's memory of the world

  memory ← UPDATE-MEMORY(memory, percept)
  action ← CHOOSE-BEST-ACTION(memory)
  memory ← UPDATE-MEMORY(memory, action)
  return action
```



- A vacuum-cleaner world with just two locations.



- Partial tabulation of a simple agent function for the vacuum-cleaner world

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

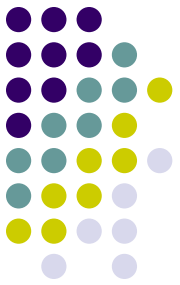
Rationality



- An ideal **rational agent** is the one that does right thing-for each possible percept sequence, do whatever actions that will maximize its **performance measure** based on
 - (1) the percept sequence, and
 - (2) its built-in and acquired knowledge.
- *It is better to design performance measures according to what one actually wants in the environment, rather than according to how one thinks the agent should behave.*
- Rationality depends on
 - The performance measure that defines the criterion of success.
 - The agent's prior knowledge of the environment.
 - The actions that the agent can perform.
 - The agent's percept sequence to date.

Based on above points the rational agent can be defined as follows:

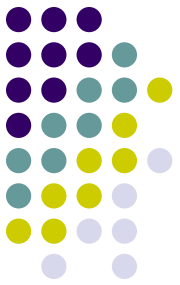
- For each possible percept sequence a rational agent should select an action that is expected to maximize its performance measure, with the evidence provided by the percept sequence and agents built-in knowledge.



Rational agents : Few properties for enabling rationality

- Rationality is not the same as perfection. Rationality maximizes expected performance, whereas perfection maximizes actual performance. Perfection requires knowing the actual outcome.
- Information gathering is an important part of rationality.
- Performing ‘looking actions’ before doing something in order to modify future percepts is called **Information gathering**.
- **Exploration** is also another kind of Information gathering.
- After information gathering the rational agent should also **learn** from what it perceives.
- A rational agent should be **autonomous**. It should learn whatever it can to compensate for partial knowledge.

The **autonomy** of an agent is the extent to which its behaviour is determined by its own experience, rather than knowledge of designer.



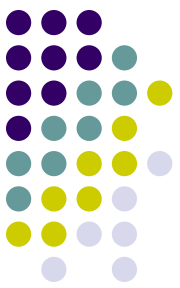
THE NATURE OF ENVIRONMENTS

- The task environment needs to be specified before designing an agent.
- PEAS(Performance, Environment, Actuators, Sensors)
- PEAS description of a task environment for Autonomous taxi

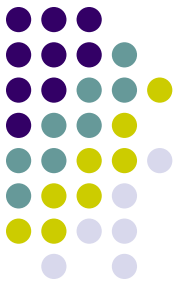
Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

- Robotics vs Softboats

Properties of task environments

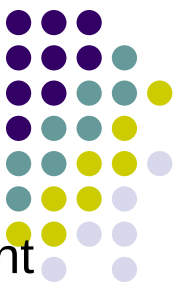


Agent Type	Percepts	Actions	Goals	Environment
Medical diagnosis system	Symptoms, findings, patient's answers	Questions, tests, treatments	Healthy patient, minimize costs	Patient, hospital
Satellite image analysis system	Pixels of varying intensity, color	Print a categorization of scene	Correct categorization	Images from orbiting satellite
Part-picking robot	Pixels of varying intensity	Pick up parts and sort into bins	Place parts in correct bins	Conveyor belt with parts
Refinery controller	Temperature, pressure readings	Open, close valves; adjust temperature	Maximize purity, yield, safety	Refinery
Interactive English tutor	Typed words	Print exercises, suggestions, corrections	Maximize student's score on test	Set of students



Properties of Task Environment

- Fully observable and partially observable
- Deterministic and stochastic
- Episodic and Sequential
- Static and Dynamic
- Discrete and Continuous
- Single agent and Multi-agent

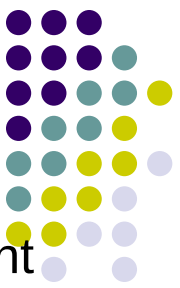


● **Fully observable vs. partially observable**

- Fully observable if the sensors detect all aspects that are relevant to the choice of action; relevance, in turn, depends on the performance measure.
- Fully observable environments are convenient
- Partially observable because of noisy and inaccurate sensors or because parts of the state are simply missing from the sensor data(data-for example, a vacuum agent with only a local dirt sensor cannot tell whether there is dirt in other squares, and an automated taxi cannot see what other drivers are thinking.)
- Game of crossword or chess-fully observable
- Driving on a hair pin bend-partially observable

● **Single agent vs. multiagent**

- Competitive
- Cooperative
- Communication
- Randomized Behavior

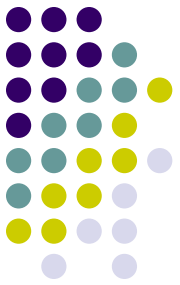


● **Deterministic/ Stochastic**

- An environment is deterministic if the next state of the environment is completely determined by the current state of the environment and the action of the agent.
- In an accessible and deterministic environment the agent need not deal with uncertainty.
- Taxi driving is clearly stochastic in this sense, because one can never predict the behavior of traffic or when the tyre would puncture.
- If the environment is deterministic except for the actions of other agents, we say that the environment is strategic.

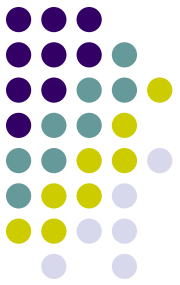
● **Episodic/ Sequential**

- An episodic environment means that subsequent episodes do not depend on what actions occurred in previous episodes.
- Such environments do not require the agent to plan ahead.



● Static/ Dynamic

- A static environment does not change while the agent is thinking.
- In a static environment the agent need not worry about the passage of time while he is thinking, nor does he have to observe the world while he is thinking.
- In static environments the time it takes to compute a good strategy does not matter.
- Dynamic environments, on the other hand, are continuously asking the agent what it wants to do
- Taxi driving is dynamic: the other cars and the taxi itself keep moving while the driving agent is deciding what to do next. Crossword puzzles are static.

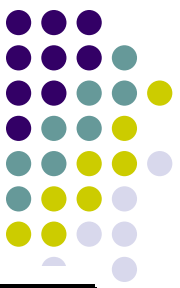


● Discrete vs. continuous

- If the number of distinct percepts and actions is limited the environment is discrete, otherwise it is continuous.
- For example, a discrete-state environment such as a chess game has a finite number of distinct states. Chess also has a discrete set of percepts and actions.
- Taxi driving is a continuous state and continuous-time problem: the speed and location of the taxi and of the other vehicles sweep through a range of continuous values and do so smoothly over time.

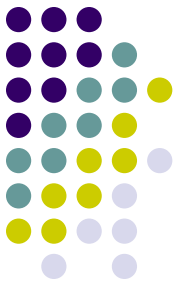
● Known vs. unknown

- This distinction refers not to the environment itself but to the agent's (or designer's) state of knowledge about the “laws of physics” of the environment.



Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

THE STRUCTURE OF AGENTS



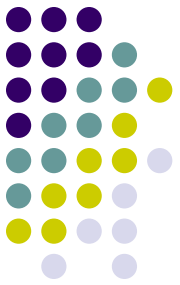
- The agent program implements the agent function. It runs on some computing device with physical sensors and actuators. This is called architecture. Thus

$$\text{agent} = \text{architecture} + \text{program}$$

- **Table-driven agents**

- Use a percept sequence/ action table in memory to find the next action. They are implemented by a (large) lookup table.
 - (a) no physical agent in this universe will have the space to store the table.
 - (b) the designer would not have time to create the table

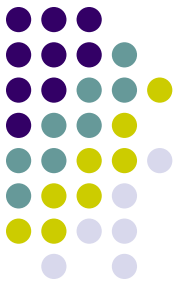
THE STRUCTURE OF AGENTS



(c) no agent could ever learn all the right table entries from its experience

(d) even if the environment is simple enough to yield a feasible table size, the designer still has no guidance about how to fill in the table entries.

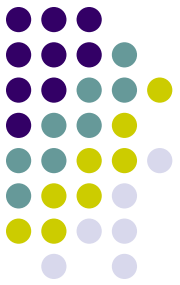
- The key AI challenge is to design programs that produce rational behavior from a small amount of code rather than indexing into large number of table entries for determining agent action.
- There are four basic kinds of agent programs which can be used for designing intelligent systems.



Basic kinds of agent programs

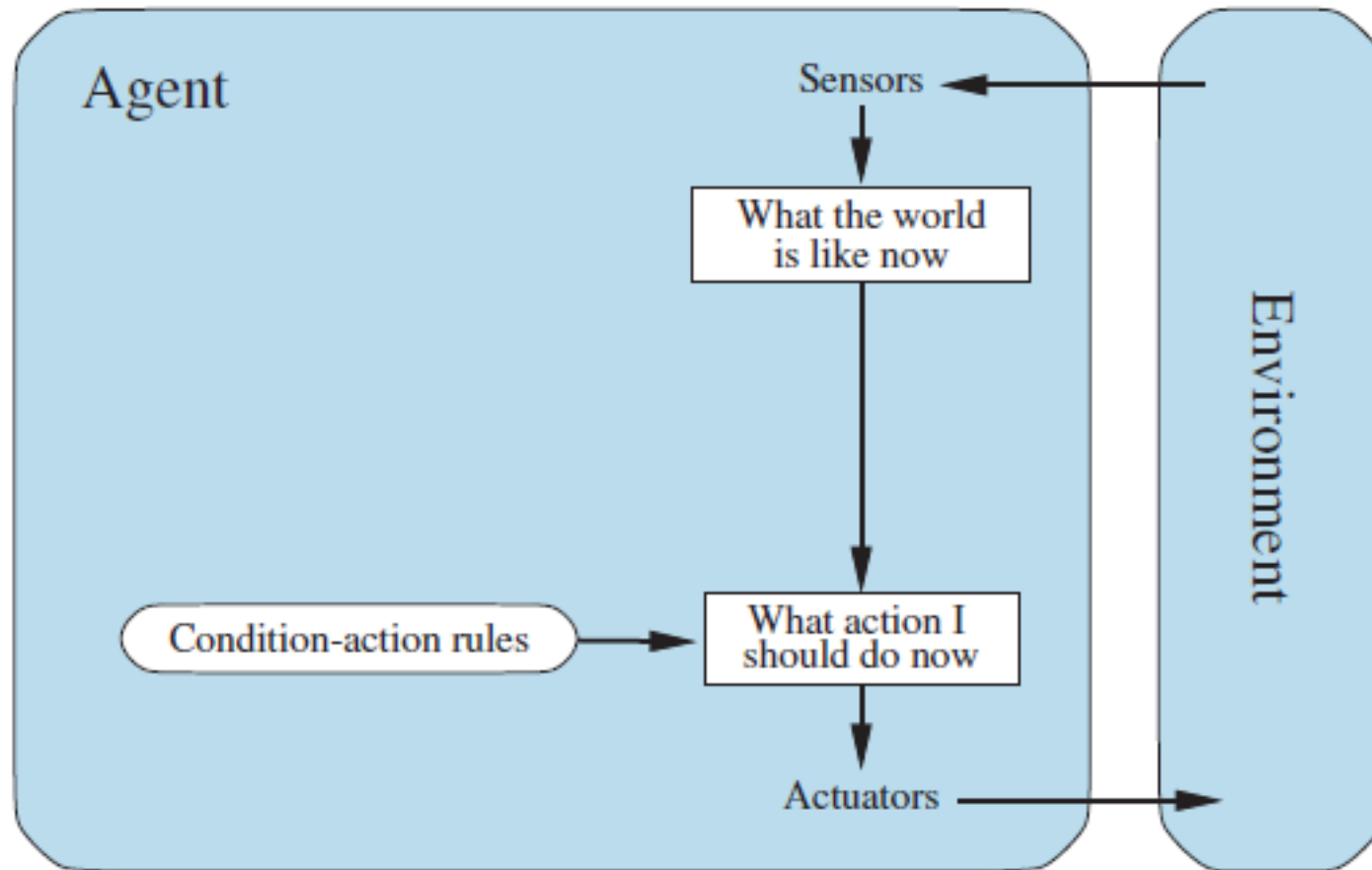
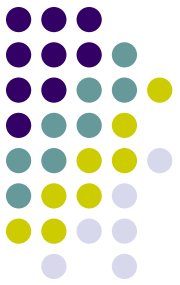
- Simple reflex agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents
- Learning agents

Simple Reflex Agent

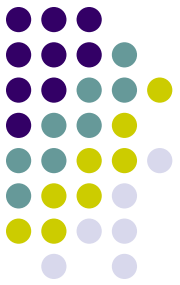


- These agents select actions on the basis of **current percept, ignoring the rest of percept history**. Ignoring the percept history cuts down the number of possibilities.
- In case of automated taxi, if the car in front brakes, then the automated taxi has to apply brakes. The human eye blinks if something approaches near it.
- These agents follow a **condition-action** rule. This rule allows the agent to make the connection from percept to action.
- Rectangles denote current internal state of agent's decision process. The ovals represent background information.
- These agents will function if correct decisions can be made on basis of current percept only. It means that environment should be **fully observable**.

Simple Reflex Agent



Simple Reflex Agent Program



```
function Simple-Reflex Agent(percept) returns an action
  static: rules, a set of condition action rules
  state  $\leftarrow$  Interpret-Input (percept)
  rule  $\leftarrow$  Rule-Match (state, rules)
  Action  $\leftarrow$  Rule-Action[rule]
  return action
```

The **Interpret-Input** function generates an abstract description of current state from the percept.

The **Rule-Match** returns the first rule that matches the given state description. {if ***car-in-front-is-braking*** then ***initiate-braking***}.

- Such agents have limited intelligence.
- **They go into infinite loop for partially observable environments.**
To escape from it, the agents can randomize their actions.

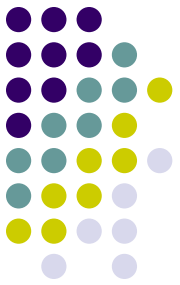
Model Based Reflex Agent



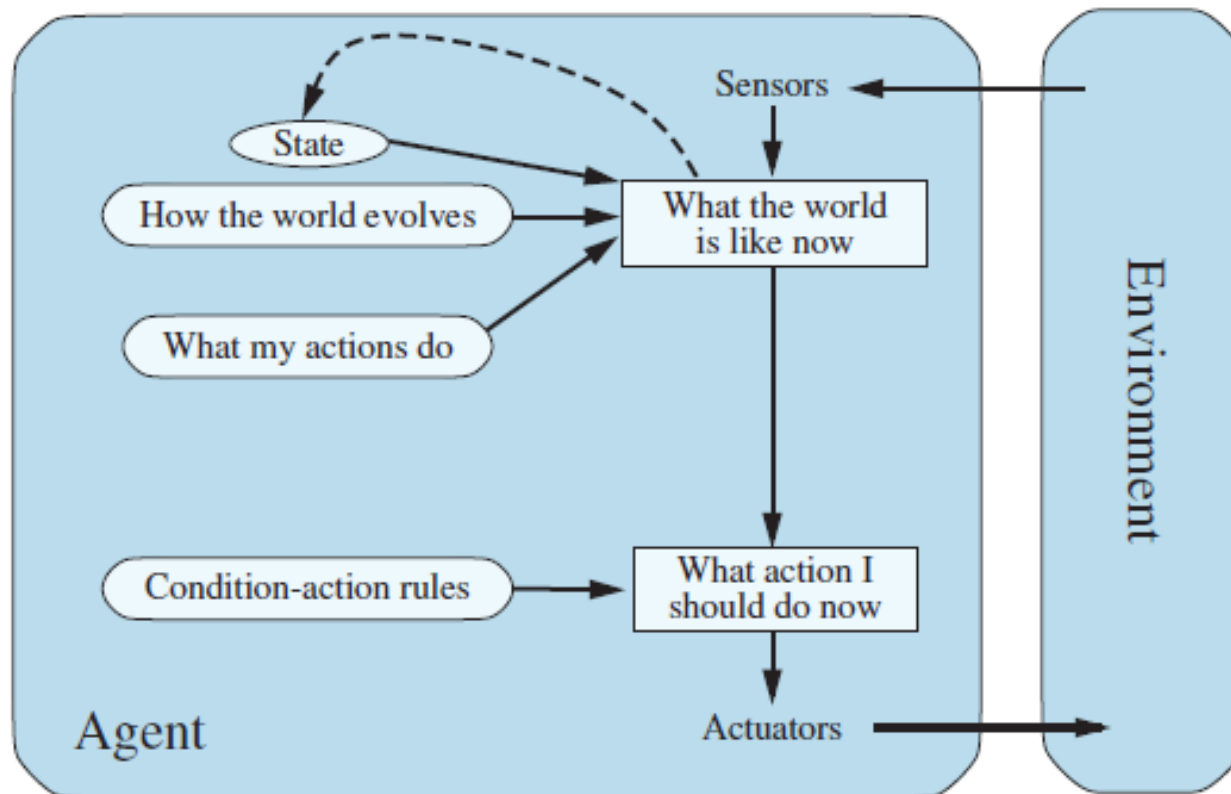
- When simple reflex agents operate in partially observable environments they can go in an infinite loop.
- To handle partial observable environments, the agent can keep track of the part of the world it can't see now. **It maintains a kind of internal state that depends on the percept history and thereby reflects some of the unobserved aspects of the current state.**
- To Update this internal state information the agent program requires to encode two kinds of knowledge.
 - How the world evolves independently of the agent. (1)
 - How the agent's own actions affect the world. (2)

E.g. For autonomous taxi: An overtaking car will be more near to the autonomous taxi as time passes. (1)

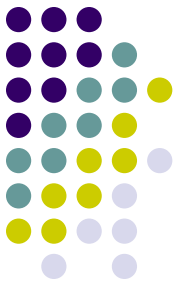
When the autonomous taxi is running on the road its position will continue to change with respect to other cars. (2)



Model Based Reflex Agent



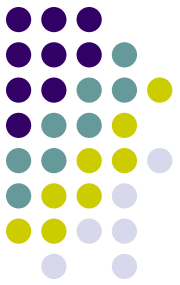
Model Based Reflex Agent



- This knowledge about how the world works is called a model of the world. An agent which uses such a model is called a model-based agent.
- The agent program can be implemented as follows:

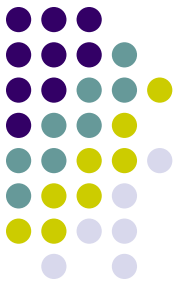
function Reflex-Agent-with-State(percept) **returns** an action
 static: state, *a description of current world state*
 rules, *a set of condition action rules*
 action, *the most recent action*
 state $\leftarrow$ Update-State(state, action, percept)
 rule $\leftarrow$ Rule-Match (state, rules)
 Action $\leftarrow$ Rule-Action[rule]
 return action

The function Update-State is responsible for creating the new internal state description.

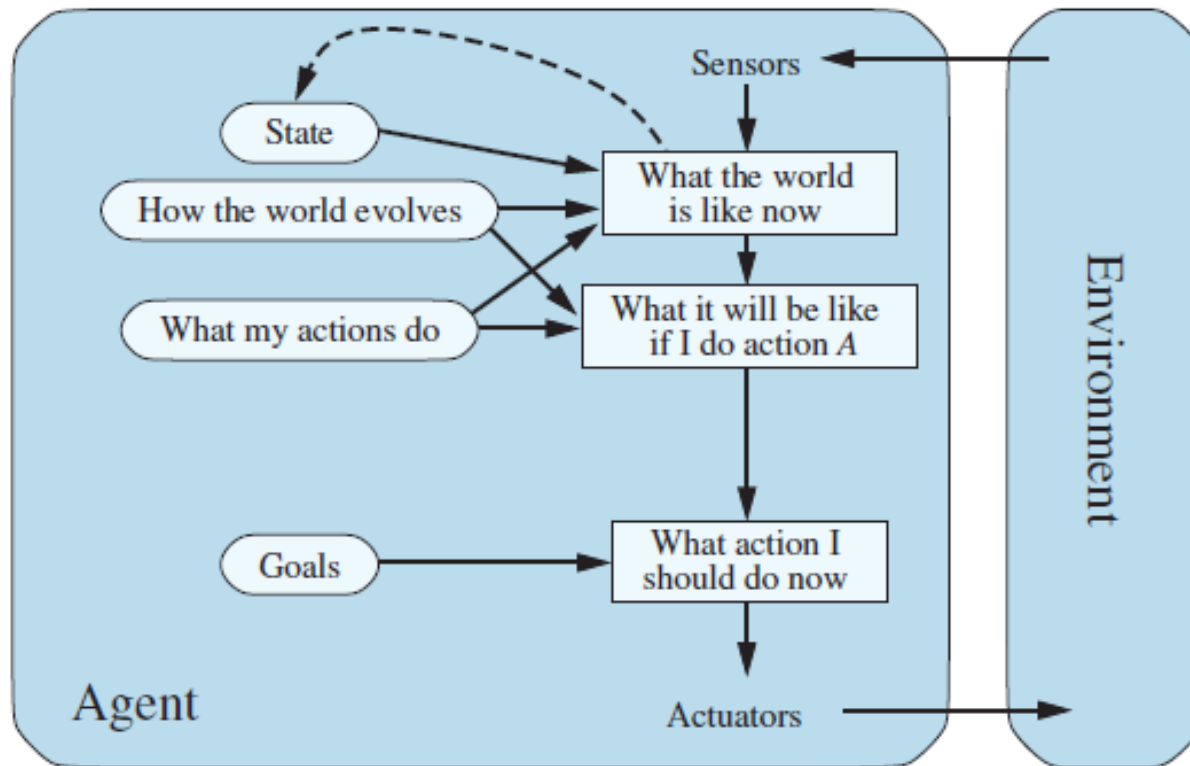


Goal based agent

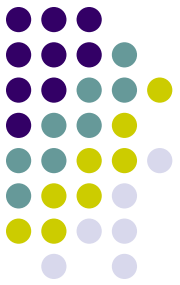
- Knowing about the current state of information is not always enough to decide what to do. The agent also needs **goal** information to describe desirable options/situations. (Taxi can go left, right, straight)
- Sometimes goal based action selection is straightforward. In other case the agent has to consider long sequence of events to find the goal. This requires elaborate **search** and **planning** techniques.
- **The goal-based agent appears less efficient but it is more flexible.** It can alter its behavior to suit new conditions. It uses knowledge about a goal to guide its actions
- For e.g. The reflex agent brakes when it sees red light. A goal based agent will know that along with braking it is also important not to hit the car.



Goal based agent

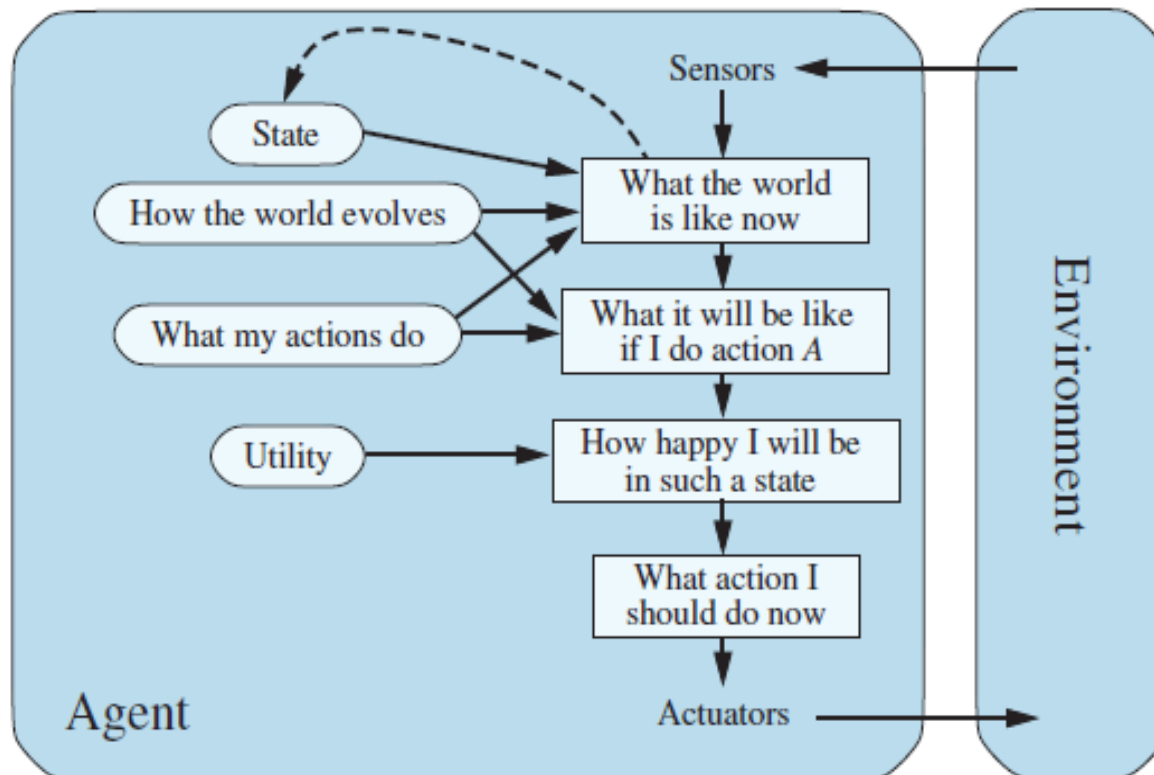
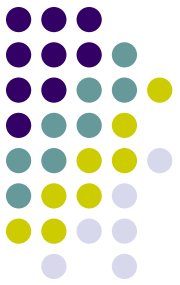


Utility based Agents

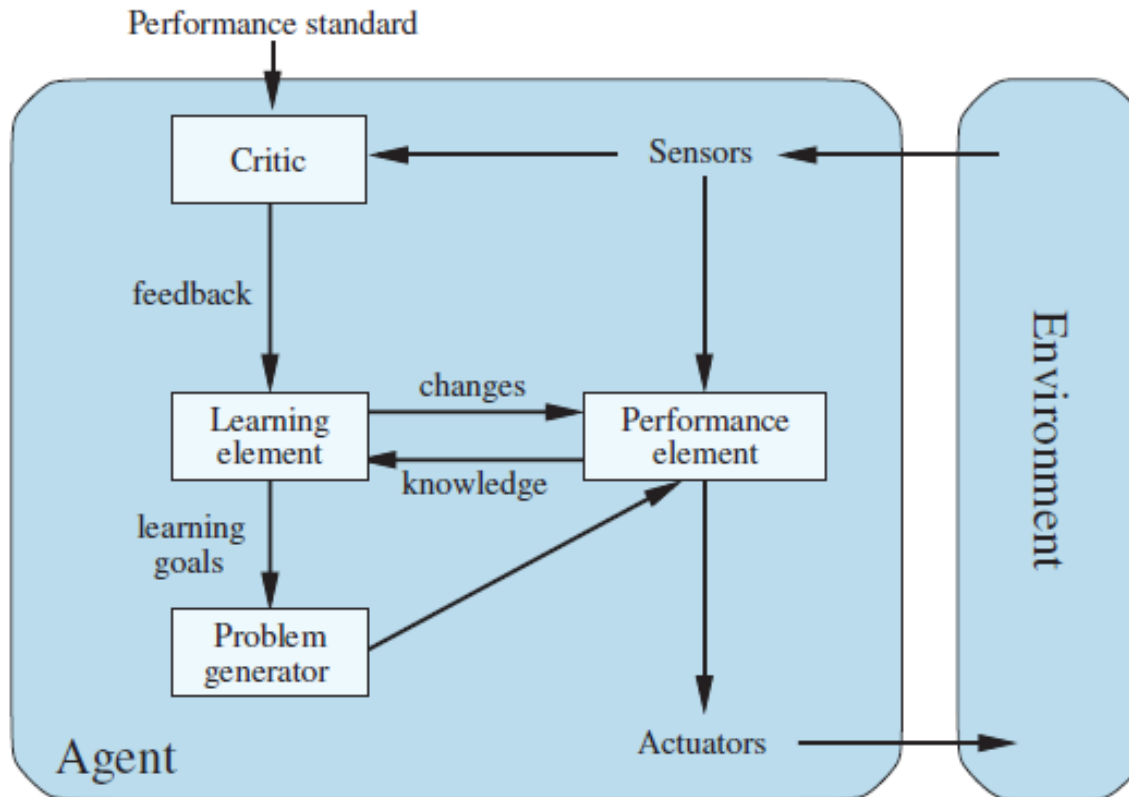
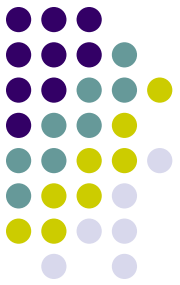


- Goals alone are not enough to generate high quality behavior in most environments. **There could be many action sequences that would generate similar goals.** Some are quicker, safer, reliable or cheaper than the others and therefore should be more desired than others.
- If one goal state is preferred to the other then it has higher utility for the agent.
- A utility function maps a state onto a real number which describes the associated degree of happiness. This function can also specify appropriate trade-offs in case of conflicting or inadequate goals.
- A rational agent must try to maximize the expected value of the utility function.

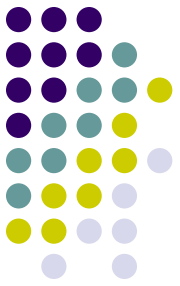
Utility based Agents



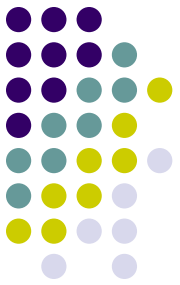
A general model of learning agents



Learning Agents



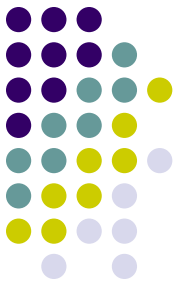
- A Learning agent can be divided into four conceptual components:
 - Learning element
 - Performance element
 - Critic
 - Problem generator
- The learning element is responsible for making improvements. It can make changes to any knowledge component.
- The Performance element is responsible for selecting external actions.
- The critic gives feedback to the learning element in terms of how well the agent is performing with respect to fixed performance parameters. The critic observes the world and passes information to learning agent.



Learning Agents

- The Problem generator suggests actions that will lead to new informative experiences. It suggests exploratory actions.
- Concept of reward (and penalty) provide direct feedback on quality of agent's behavior.
- For e.g. Taxi driver makes quick right turn across 3 lanes
 - The Critic observes shocking language by passenger and other drivers and informs it to the Learning element about the bad action.
 - Learning element tries to modify performance elements for future
 - Problem generator suggests to experiment out something called Brakes on different Road conditions

To build a system to solve a problem

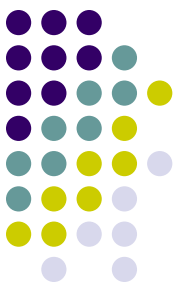


- Define the problem precisely
- Analyze the problem
- Isolate and represent the task knowledge that is necessary to solve the problem
- Choose the best problem-solving technique and apply it to the particular problem.

Defining the problem as State Space Search



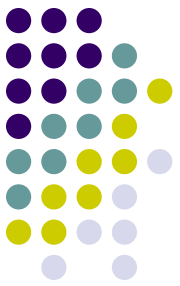
- The state space representation forms the basis of most of the AI methods.
- A problem is really a collection of information that the agent will use to decide what to do.
- **State Space Representation:**
 1. The initial state
 2. Goal State
 3. Operator / Successor Function
 4. The description of an action in terms of which state will be reached by carrying out the action in a particular state.
 5. Goal Test
 6. Path cost



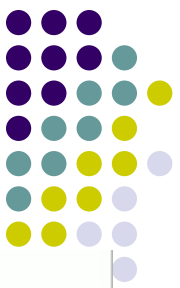
```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state  $\leftarrow$  UPDATE-STATE(state, percept)
  if seq is empty then
    goal  $\leftarrow$  FORMULATE-GOAL(state)
    problem  $\leftarrow$  FORMULATE-PROBLEM(state, goal)
    seq  $\leftarrow$  SEARCH(problem)
    if seq = failure then return a null action
  action  $\leftarrow$  FIRST(seq)
  seq  $\leftarrow$  REST(seq)
  return action
```

-
- It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over.

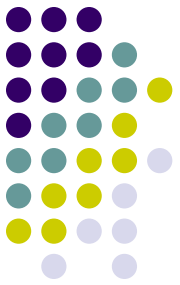


- Its structure corresponds to the structure of problem solving in two important ways:
 - It allows for a formal definition of a problem as the need to **convert some given situation into some desired situation using a set of permissible operations**.
 - It permits us to define the process of solving a particular problem as a combination of **known techniques** (each represented as a rule defining a single step in the space) and **search** (the general technique of exploring the space to try to find some path from current state to a goal state).
 - Search is a very important process in the solution of hard problems for which no more direct techniques are available.



General Process followed in solving problem using State Space Search

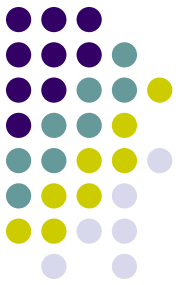
1. First, select some way to represent states in the given problem in an unambiguous way.
2. Next, formulate all actions or operators that can be performed in states, including their preconditions and effects.
 - Actions or operates are called **PRODUCTION RULES**
3. Represent the initial state or states of the problem.
4. Formulate precisely when a state satisfies the goal of our problem.
5. Activate the production rules on the initial state and its descendants, until a goal state is reached.



Example problems

- The problem-solving approach can be applied to vast number of task environments. Some problems are toy problems and some are real world problems.
- Toy problems are used to illustrate and test various problem-solving methods. They can also be used to compare various algorithms.
 - 8-queens problem (incremental v/s complete-state formulation)
- Real world problems require solutions which are possible and feasible in real world.
 - The travelling salesman problem
 - VLSI layout problem
 - Robot Navigation

The water jug problem (A formal description of the problem)

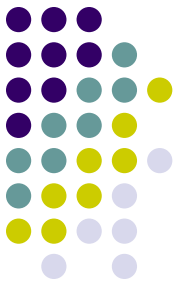


There are two jugs, one 4 litre and other 3 litre with no measuring marks on it. A pump can be used to fill the jugs with water. How can you get exactly 2 litres of water in the 4-litre jug?

The above toy problem can be solved by the steps enumerated below.

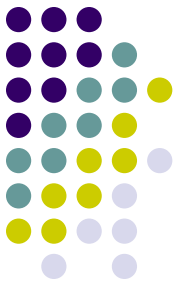
1. Define a state space that contains all possible configurations of the relevant objects.
2. Specify **one** or more states within that space that describe situations from which the problem solving may start. These states are called the initial states.
3. Specify one or more states that would be acceptable as solutions to the given problem. These are the goal states.
4. Specify the set of rules that describe the actions (also called operators) available. The problem can then be solved by applying these rules with an appropriate control strategy by moving through the problem space until a path from the initial state to the goal state is found.

The water jug problem contd..



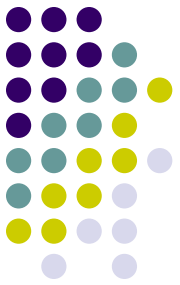
1. Define a state space that contains all possible configurations of the relevant objects.
 - The state space for this problem can be described as a set of ordered pair of integers $[x, y]$ where $x=0,1,2,3,4$ (5 states) and $y=0,1,2,3$ (4 states). **Total possible states=5*4=20**
 - Not all these states are possibly reachable from the starting point depending on the rules.
2. Specify one or more states within that space that describe situations from which the problem solving may start. These states are called the initial states.
 - The start state is $(0, 0)$: both jugs are empty.
3. Specify one or more states that would be acceptable as solutions to the given problem. These are the goal states.
 - The goal state is $(2,n)$: it can be for any value of n since problem does not specify how much water should be in 3-litre jug.
4. Specify the set of rules that describe the actions (also called operators) available. The problem can then be solved by applying these rules with an appropriate control strategy by moving through the problem space until a path from the initial state to the goal state is found.

Possible operators for the water jug problem



Sr.	Current State	Next State	Descriptions
1	(x, y) if $x < 4$	$(4, y)$	Fill the 4 gallon jug
2	(x, y) if $y < 3$	$(x, 3)$	Fill the 3 gallon jug
3	(x, y) if $x > 0$	$(x - d, y)$	Pour some water out of the 4 gallon jug
4	(x, y) if $y > 0$	$(x, y - d)$	Pour some water out of the 3 gallon jug
5	(x, y) if $y > 0$	$(0, y)$	Empty the 4 gallon jug
6	(x, y) if $y > 0$	$(x, 0)$	Empty the 3 gallon jug on the ground
7	(x, y) if $x + y \geq 4$ and $y > 0$	$(4, y - (4 - x))$	Pour water from the 3 gallon jug into the 4 gallon jug until the 4 gallon jug is full
8	(x, y) if $x + y \geq 3$ and $x > 0$	$(x - (3 - y), 3)$	Pour water from the 4 gallon jug into the 3-gallon jug until the 3 gallon jug is full
9	(x, y) if $x + y \leq 4$ and $y > 0$	$(x + y, 0)$	Pour all the water from the 3 gallon jug into the 4 gallon jug
10	(x, y) if $x + y \leq 3$ and $x > 0$	$(0, x + y)$	Pour all the water from the 4 gallon jug into the 3 gallon jug
11	$(0, 2)$	$(2, 0)$	Pour the 2 gallons from 3 gallon jug into the 4 gallon jug
12	$(2, y)$	$(0, y)$	Empty the 2 gallons in the 4 gallon jug on the ground

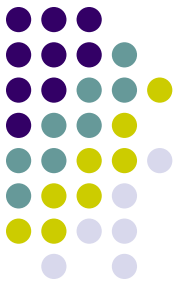
One Possible solution for the water jug problem



Gallons in the 4-gallon jug	Gallons in the 3-gallon jug	Rule applied
0	0	2
0	3	9
3	0	2
3	3	7
4	2	5 or 12
0	2	9 or 11
2	0	...

These two steps
required only if 2 Lt is
to be measured in
the 4 Lt jug





The 8-puzzle

- **States:** a state description specifies the location of each of the eight tiles in one of the nine squares. For efficiency, it is useful to include the location of the blank.
- **Initial State:** any state
- **Operators:** blank moves left, right, up, or down
- **Transition model:** Given a state and action, this returns the resulting state; for example, if we apply *Left* to the start state in Figure the resulting state has the 5 and the blank switched.
- **Goal test:** state matches the goal configuration shown in Figure
- **Path cost:** each step costs 1, so the path cost is just the length of the path.

7	2	4
5		6
8	3	1

Start State

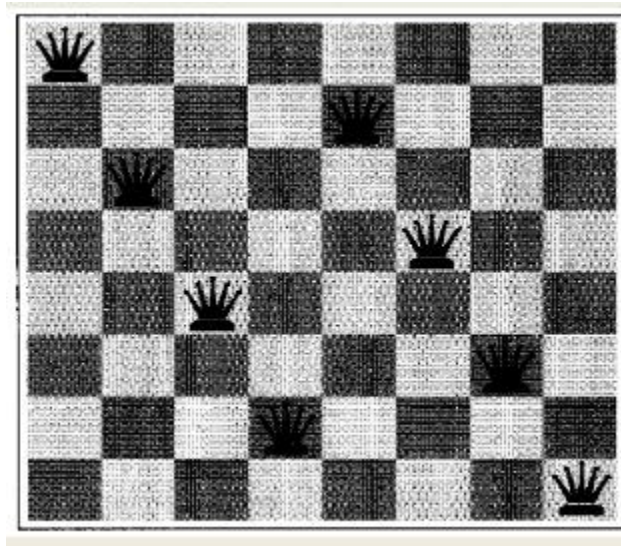
	1	2
3	4	5
6	7	8

Goal State

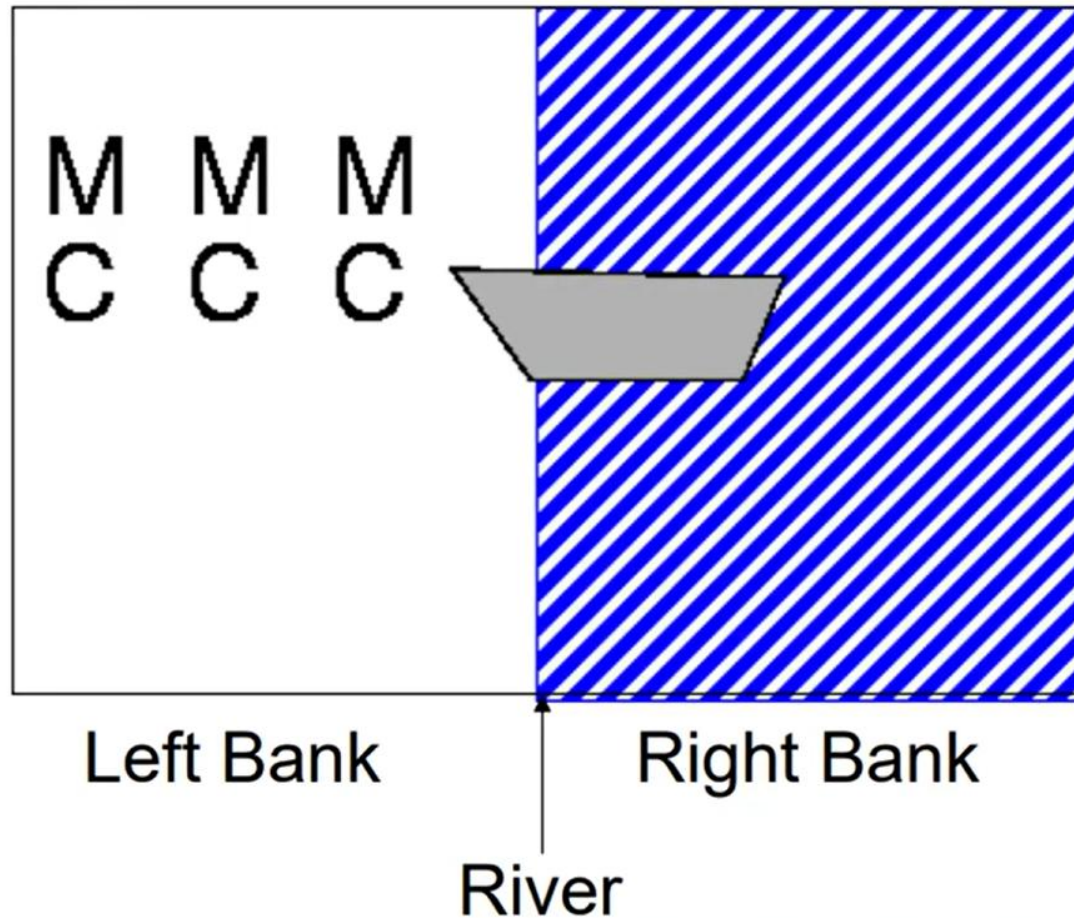
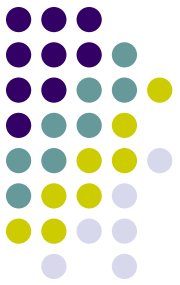
The 8-queens problem

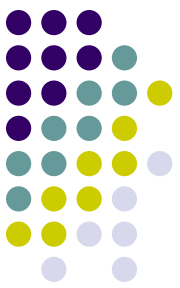


- **States:** Any arrangement of 0 to 8 queens on the board is a state.
- **Initial state:** No queens on the board.
- **Actions/Operators:** Add a queen to any empty square.
- **Transition model:** Returns the board with a queen added to the specified square.
- **Goal test:** 8 queens are on the board, none attacked

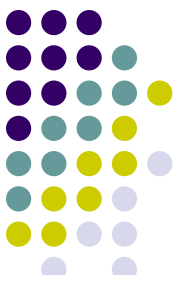


Missionaries and Cannibals Problem





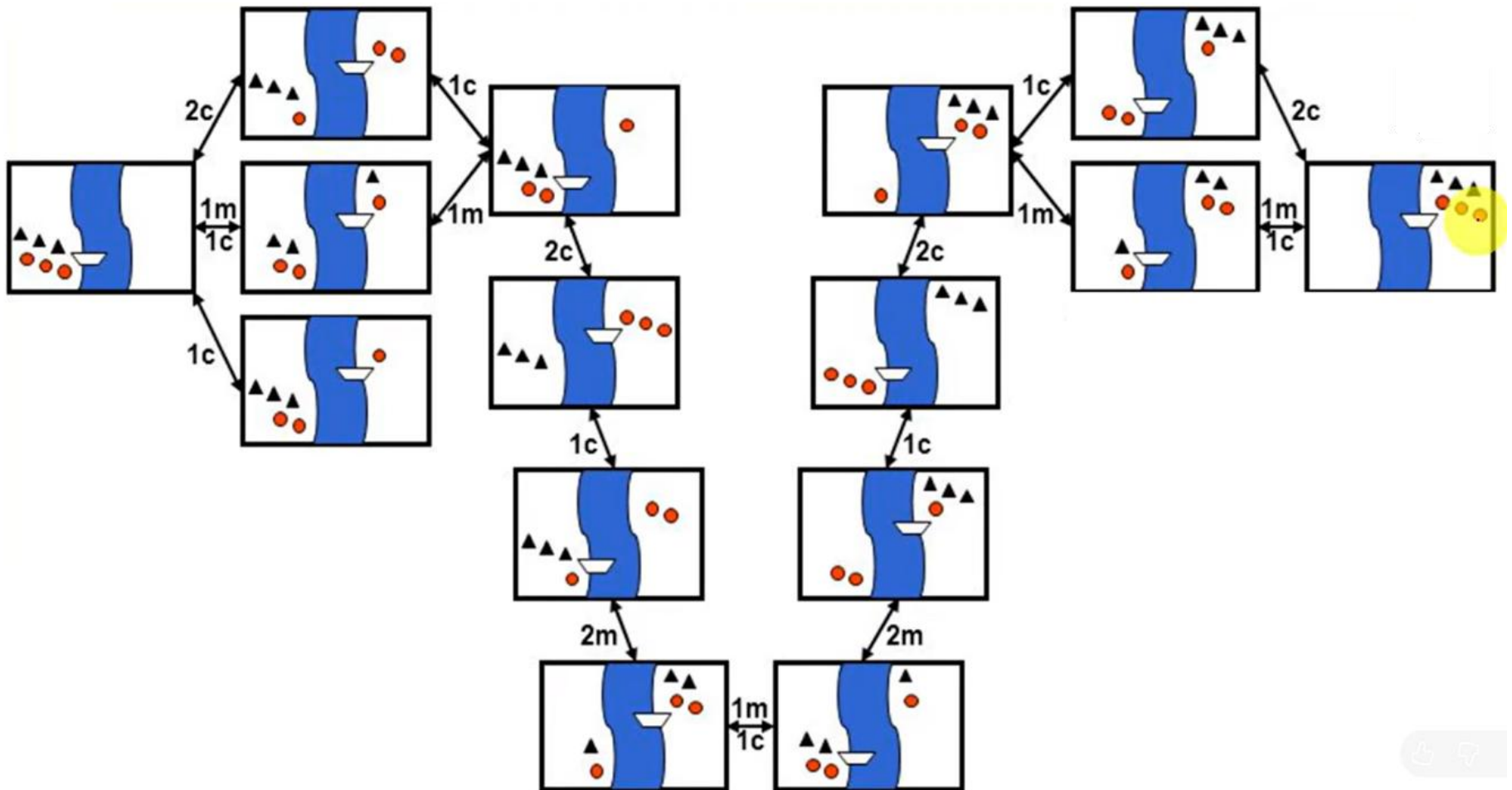
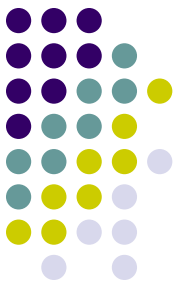
- Three missionaries and three cannibals wish to cross the river.
- They have a small boat that will carry up to two people.
- Everyone can navigate the boat.
- If at any time the Cannibals outnumber the Missionaries on either bank of the river, they will eat the Missionaries.
- Find the smallest number of crossings that will allow everyone to cross the river safely.

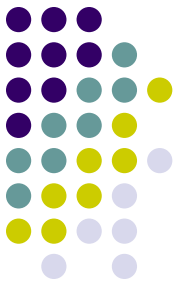


- **States:** combination of missionaries and cannibals and boat on each side of river.
- **Initial State:** 3 missionaries, 3 cannibals and the boat are on the near bank
- **Successor Function:** Move the boat containing some set of occupants across the river (in either direction) to the other side



- **Constraint:** Missionaries can never be outnumbered by cannibals on either side of river, or else the missionaries are killed or cannibals will eat the missionaries.
- **Action:** Ride the boat with maximum two persons (one or two) across the river (in either direction) to the other side
- **Goal Test:** Move all the missionaries and cannibals across the river.
- **Path cost:** Requires minimum number of moves





Happy Learning!!!!